

Sprout Habits — Complete Codebase Guide

Version 1.0.0 · Written for developers with Python experience

Table of Contents

1. What This App Is
2. Tech Stack — The Python Analogy
3. Project Structure
4. Core Concepts: React & TypeScript for Python Developers
5. Data Models — The Blueprint of Everything
6. Local Storage Layer
7. Supabase — The Cloud Database
8. Authentication (`useAuth`)
9. The Brain of the App — `useAppState`
10. Habit Logic Library (`lib/habits.ts`)
11. Badges Library (`lib/badges.ts`)
12. Shop Items Library (`lib/shopItems.ts`)
13. App Entry Point & Routing
14. Layout & Navigation
15. Page: Home (`Index.tsx`)
16. Page: Calendar (`CalendarPage.tsx`)
17. Page: Friends (`FriendsPage.tsx`)
18. Page: Shop / Character (`CharacterPage.tsx`)
19. Page: AI (`AIPage.tsx`)
20. Page: Profile (`ProfilePage.tsx`)
21. Page: Upgrade (`UpgradePage.tsx`)
22. Page: Auth (`AuthPage.tsx`)
23. Page: Notifications (`NotificationsPage.tsx`)
24. Component: HabitCard
25. Component: CreateHabitDialog
26. Component: CoachPanel
27. Component: StreakRecoveryCard
28. Component: Heatmap
29. Component: OnboardingFlow
30. Component: SplashScreen
31. Engagement Systems
32. Monetization Logic
33. Theming System
34. How Data Flows End-to-End
35. Glossary

1. What This App Is

Sprout Habits is a **mobile-first habit tracking Progressive Web App (PWA)**. A PWA is a website that behaves like a phone app — it can be installed on a home screen, can send push notifications, and works on any device with a browser.

Users can:

- Create habits and mark them complete each day
- Build streaks and earn coins as rewards
- Spend coins in a virtual shop to unlock character items
- See detailed analytics about their consistency
- Compete with friends on a leaderboard
- Chat with an AI coach that analyzes their real habit data
- Receive browser notifications when they're about to lose a streak
- See a weekly recap every Saturday

The app is monetized through a **Pro subscription (\$4.99/month)** that unlocks AI features, unlimited habits, and auto-streak-freeze.

2. Tech Stack — The Python Analogy

If you know Python, here's how to map the technology stack:

Python world	This app's equivalent	Purpose
Python script	TypeScript (<code>.ts/.tsx</code>)	The programming language
Type hints (<code>x: int</code>)	TypeScript types (<code>x: number</code>)	Static type checking
Flask / FastAPI	React + React Router	Renders the UI, handles navigation
Jinja2 templates	JSX (HTML written inside JS)	Defines what the UI looks like
<code>pip install</code>	<code>npm install</code>	Package manager
<code>dict, list</code>	<code>object, array</code>	Data structures
PostgreSQL + psycopg2	Supabase (PostgreSQL + SDK)	Cloud database
<code>localStorage</code> (no Python equivalent)	browser <code>localStorage</code>	Offline data cache
Flask session / JWT	Supabase Auth	User authentication
Django settings.py	<code>settings.json</code> / <code>.env</code>	App configuration
CSS framework	Tailwind CSS	Styling
Component library	shadcn/ui	Pre-built UI pieces

The key mental shift: In Python you run a script top-to-bottom. In React, you write **components** — functions that return UI. When data changes, React automatically re-runs those functions and updates only the parts of the screen that changed. You don't manually update the DOM.

3. Project Structure

```
Habit Bloom App/
├── src/
│   ├── App.tsx          ← Entry point, routing, providers
│   ├── main.tsx         ← Mounts React into the HTML page
│   └── pages/           ← One file per screen
│       ├── Index.tsx    ← Home screen
│       ├── CalendarPage.tsx ← Month calendar view
│       ├── FriendsPage.tsx ← Friends league + challenges
│       ├── CharacterPage.tsx ← Shop / wardrobe
│       ├── AIPage.tsx    ← AI chat + coach
│       ├── ProfilePage.tsx ← Settings and habit management
│       ├── UpgradePage.tsx ← Pro subscription page
│       ├── AuthPage.tsx  ← Sign in / sign up
│       ├── CoachPage.tsx ← Standalone coach page
│       ├── NotificationsPage.tsx ← Notification preferences
│       └── NotFound.tsx  ← 404 page
│
│   ├── components/      ← Reusable UI pieces
│       ├── AppLayout.tsx ← Wraps all pages with bottom nav
│       ├── HabitCard.tsx ← Individual habit row
│       ├── CreateHabitDialog.tsx ← Create/edit habit modal
│       ├── CoachPanel.tsx ← AI coach UI (reused in 2 places)
│       ├── StreakRecoveryCard.tsx ← "Streak at risk" recovery UI
│       ├── Heatmap.tsx   ← GitHub-style activity grid
│       ├── OnboardingFlow.tsx ← First-launch setup wizard
│       ├── SplashScreen.tsx ← Loading animation on first visit
│       └── CelebrationOverlay.tsx ← Confetti animation on unlocks
│
│   ├── hooks/           ← Shared logic (React's version of a module)
│       ├── useAppState.tsx ← ALL app data + business logic
│       ├── useAuth.tsx    ← Authentication state
│       └── useTheme.tsx   ← Dark/light/system theme
│
│   ├── lib/             ← Pure helper functions (no UI)
│       ├── types.ts      ← TypeScript type definitions
│       ├── habits.ts     ← Streak math, scheduling logic
│       ├── badges.ts     ← Badge unlock conditions
│       ├── shopItems.ts  ← Shop item catalog
│       ├── storage.ts    ← localStorage read/write
│       ├── celebrate.ts  ← Confetti trigger
│       └── utils.ts      ← cn() helper for class names
│
│   └── integrations/supabase/
│       └── client.ts      ← Supabase connection
```

└─ types.ts	← Auto-generated DB type definitions
└─ package.json	← Dependencies (like requirements.txt)
└─ vite.config.ts	← Build tool configuration
└─ tailwind.config.ts	← CSS utility configuration
└─ tsconfig.json	← TypeScript compiler settings

4. Core Concepts: React & TypeScript for Python Developers

4.1 Components

A React component is just a function that returns HTML-like syntax called JSX:

```
// Python function
def greet(name: str) -> str:
    return f"<h1>Hello, {name}</h1>"

// React component (TypeScript)
const Greet = ({ name }: { name: string }) => {
    return <h1>Hello, {name}</h1>;
};
```

The `{ name }` syntax is called **destructuring** — it's like `name = kwargs['name']` in Python. The `{ name: string }` part is the TypeScript type annotation.

4.2 State — `useState`

In Python, variables just hold values. In React, if you want the UI to update when a variable changes, you use `useState`:

```
// Python equivalent concept:
# count = 0
# def increment():
#     global count
#     count += 1
#     re_render_ui() # React does this automatically

// React way:
const [count, setCount] = useState(0);
// count = current value
// setCount = function to change it (triggers re-render)

// Usage:
<button onClick={() => setCount(count + 1)}>
    Clicked {count} times
</button>
```

Every time `setCount` is called, React re-runs the component function and updates the screen.

4.3 Effects — `useEffect`

`useEffect` runs code after the component renders. Think of it as an event listener that triggers on specific conditions:

```
// Runs once when the component first appears (like __init__)
useEffect(() => {
  console.log("Component mounted");
}, []); // empty array = run once

// Runs whenever `userId` changes (like a property setter)
useEffect(() => {
  fetchUserData(userId);
}, [userId]); // dependency array

// Runs after every render (no dependency array)
useEffect(() => {
  document.title = `${habits.length} habits`;
});
```

The function returned from `useEffect` is a **cleanup function** — it runs when the component unmounts or before the effect runs again (like a destructor in Python).

4.4 Context — `useContext`

Context is React's way of sharing data across many components without passing it down manually at every level. Think of it as a global dictionary:

```
# Python analogy
APP_STATE = {} # global dict all modules can import

# React equivalent
const AppCtx = createContext(null); // the "global dict"

// Provider: sets the value
<AppCtx.Provider value={{ habits, user, addHabit }}>
  {children}
</AppCtx.Provider>

// Consumer: reads the value from anywhere in the tree
const { habits, addHabit } = useContext(AppCtx);
```

4.5 Custom Hooks

A custom hook is just a function that starts with `use` and can call other hooks. It's the React equivalent of a Python class that manages state:

```
# Python class approach
class AppState:
    def __init__(self):
        self.habits = []
    def add_habit(self, habit):
        self.habits.append(habit)

# React hook approach
const useAppState = () => {
  const [habits, setHabits] = useState([]);
  const addHabit = (habit) => setHabits(prev => [...prev, habit]);
  return { habits, addHabit };
};
```

4.6 TypeScript Types

TypeScript adds type checking to JavaScript, similar to Python's type hints but enforced at compile time:

```
// Python
def add_habit(name: str, days: list[int]) -> None:
    pass

// TypeScript
const addHabit = (name: string, days: number[]): void => {
  // ...
};

// Python dataclass
@dataclass
class Habit:
    id: str
    name: str
    emoji: str

// TypeScript interface
interface Habit {
    id: string;
    name: string;
    emoji: string;
}
```

4.7 The `cn()` Helper

Throughout the code you'll see `cn(...)` used for CSS classes. It's a utility that intelligently merges Tailwind CSS classes:

```
// Without cn: messy string concatenation
className={"rounded-xl p-4 " + (isActive ? "bg-primary text-white" : "bg-card")}
```

```
// With cn: clean and readable
className={cn("rounded-xl p-4", isActive && "bg-primary text-white")}
```

4.8 Tailwind CSS

Instead of writing CSS files, this app uses **Tailwind CSS** — pre-built utility classes applied directly in HTML:

```
<!-- Traditional CSS -->
<style>.card { border-radius: 12px; padding: 16px; background: white; }</style>
<div class="card">...</div>

<!-- Tailwind approach -->
<div class="rounded-xl p-4 bg-white">...</div>
```

Common patterns you'll see:

- `rounded-3xl` = very rounded corners
- `p-4 / px-3 py-2` = padding (all sides / horizontal & vertical)
- `flex items-center gap-3` = flexbox layout with centered items and spacing
- `text-sm font-semibold` = small, bold text
- `text-muted-foreground` = gray/secondary text
- `bg-card border border-border` = card-style box with border
- `shadow-soft` = custom subtle shadow (defined in the theme)

5. Data Models — The Blueprint of Everything

File: `src/lib/types.ts`

This file defines the shape of all data in the app. Think of these as Python dataclasses.

5.1 The `Habit` Type

```
interface Habit {
  id: string;           // UUID, e.g. "a1b2c3d4-..."
  name: string;         // "Morning Run"
  emoji: string;        // "🏃"
  color: string;        // "primary" | "accent" | "success" | "warning"
  days: WeekDay[];      // [1, 2, 3, 4, 5] = Mon-Fri
  completions: string[]; // ["2024-01-15", "2024-01-16", ...]
  createdAt: string;    // "2024-01-01"
  reminderTime?: string; // "08:00" – optional per-habit alarm
  expiresAt?: string;   // "2024-01-15" – for free-tier temp habits
}

type WeekDay = 0 | 1 | 2 | 3 | 4 | 5 | 6; // 0=Sunday, 6=Saturday
```

Key design decision: completions are stored as an array of date strings rather than a boolean "done today". This means the full history is always available for analytics, streak calculations, and heatmaps.

The `?` after a field name means it's optional (equivalent to `Optional[str]` in Python typing).

5.2 The `UserState` Type

```
interface UserState {
  coins: number;           // Virtual currency balance
  unlocked: string[];      // IDs of owned shop items, e.g. ["default",
                           "hat-leaf"]
  characterName: string;   // "Sprout" – the mascot's name
  displayName?: string;    // "Alex" – shown in greetings
  goal?: string;           // "Get fit" – used by AI coach
  onboardingDone?: boolean; // Has the user completed setup?
  reminders?: {
    enabled: boolean;
    time: string;           // "09:00"
    lastNotified?: string;  // "2024-01-15" – to avoid repeat notifications
  };
  theme?: "light" | "dark" | "system";
  isPro?: boolean;
  proSince?: string;
  streakFreezes?: string[]; // Dates the streak was "frozen"
  lastLoginDate?: string;   // For daily login bonus
  loginStreak?: number;     // 1-7 day login streak
  friendCode?: string;      // 6-char code like "A7B3K2"
}
```

5.3 The `CalendarEvent` Type

```
interface CalendarEvent {
  id: string;
  title: string;           // "Math exam"
  date: string;            // "2024-01-15"
  note?: string;           // Optional description
}
```

6. Local Storage Layer

File: `src/lib/storage.ts`

The browser's `localStorage` is a key-value store that persists between page reloads (like a simple JSON file on disk). It's the app's offline-first cache.

```
const KEYS = {
  habits: "ht_habits_v1", // versioned key prevents old data conflicts
```



```
events: "ht_events_v1",
user:   "ht_user_v1",
};
```

Why version the keys? If you change the data structure in a future update, users with old data in `ht_habits_v1` won't get errors — you'd create `ht_habits_v2` and migrate.

Reading

```
export const loadHabits = (): Habit[] => {
  try {
    return JSON.parse(localStorage.getItem(KEYS.habits) || "[]");
  } catch {
    return []; // if the JSON is corrupted, return empty array safely
  }
};
```

This is equivalent to:

```
import json, os

def load_habits():
    try:
        with open("habits.json") as f:
            return json.load(f)
    except:
        return []
```

Writing

```
export const saveHabits = (h: Habit[]) =>
  localStorage.setItem(KEYS.habits, JSON.stringify(h));
```

Default User State

The `loadUser()` function has a smart default — if there's nothing in storage, it returns a fresh user with safe defaults:

```
return {
  coins: 0,
  unlocked: ["default"], // "default" skin is always owned
  characterName: "Sprout",
  onboardingDone: false, // triggers the onboarding wizard
  reminders: { enabled: false, time: "09:00" },
```

```

    theme: "dark",
    isPro: false,
    streakFreezes: [],
    loginStreak: 0,
  };

```

7. Supabase — The Cloud Database

File: `src/integrations/supabase/client.ts`

Supabase is a hosted PostgreSQL database with a REST API and real-time subscriptions. Think of it as "PostgreSQL + Django REST Framework + authentication, all hosted for you."

```

import { createClient } from "@supabase/supabase-js";
export const supabase = createClient(SUPABASE_URL, SUPABASE_ANON_KEY);

```

The `supabase` object is the connection. It's used everywhere in the app to read/write data.

7.1 Database Tables

The app has these PostgreSQL tables:

profiles — One row per user, mirrors `UserState`:

```

id          UUID (primary key, = auth.users.id)
user_id     UUID (references auth.users)
coins       INTEGER
unlocked    TEXT[] (PostgreSQL array)
character_name TEXT
display_name TEXT
goal        TEXT
streak_freezes TEXT[]
login_streak INTEGER
last_login_date DATE
friend_code TEXT (UNIQUE)
is_pro      BOOLEAN
-- ...etc

```

habits — One row per habit:

```

id          UUID
user_id     UUID
name        TEXT
emoji       TEXT
color       TEXT
days       INTEGER[]

```

```
completions TEXT[]
reminder_time TEXT
expires_at DATE
created_at TIMESTAMPTZ
```

calendar_events — Events pinned to calendar dates.

friendships — Stores friend connections:

```
id          UUID
requester_id UUID (who sent the friend request)
addressee_id UUID (who received it)
status      TEXT ('pending' | 'accepted')
```

habit_reminders — Per-habit reminder settings.

notification_preferences — Global notification settings.

7.2 Row Level Security (RLS)

Every table has **Row Level Security** enabled. This means PostgreSQL enforces that users can only read/write their own data, even if someone bypasses the app and calls the API directly:

```
-- Example RLS policy on habits table:
CREATE POLICY "Users can manage their own habits"
ON habits
FOR ALL
USING (auth.uid() = user_id);
```

`auth.uid()` is Supabase's function that returns the currently logged-in user's ID. If this doesn't match `user_id`, the operation is rejected.

7.3 How Queries Work

```
// SELECT – reading data
const { data, error } = await supabase
  .from("habits")
  .select("*")
  .eq("user_id", authUser.id);

// UPSERT – insert or update based on primary key
const { error } = await supabase
  .from("habits")
  .upsert({ id: habit.id, user_id: authUser.id, name: "Run", ... });

// UPDATE – update existing row
const { error } = await supabase
```

```
.from("profiles")
.update({ coins: 150 })
.eq("user_id", authUser.id);

// DELETE
const { error } = await supabase
  .from("habits")
  .delete()
  .eq("id", habitId);
```

`await` is equivalent to Python's `await` in async functions. All Supabase calls are asynchronous — they go over the network and return a Promise.

7.4 Supabase Edge Functions

The AI features call **Supabase Edge Functions** — serverless functions that run on Supabase's infrastructure (like AWS Lambda):

```
const { data, error } = await supabase.functions.invoke("ai-chat", {
  body: { message: "How am I doing?", habits: summarizedHabits, ... }
});
```

The function `ai-chat` runs on Supabase's servers, calls an AI API (Google Gemini or similar), and returns the response. The API key never reaches the browser.

8. Authentication (`useAuth`)

File: `src/hooks/useAuth.tsx`

This hook manages the login state. It wraps Supabase Auth, which handles email/password sign-up, sign-in, and session management.

```
const AuthProvider = ({ children }) => {
  const [session, setSession] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    // 1. Subscribe to auth changes FIRST (to catch the initial session restore)
    const { data: sub } = supabase.auth.onAuthStateChange((_event, s) => {
      setSession(s);
    });

    // 2. Then get the current session from storage
    supabase.auth.getSession().then(({ data }) => {
      setSession(data.session);
      setLoading(false);
    });
  });

  return children;
};
```

```
    return () => sub.subscription.unsubscribe();
  }, []);

  // ...
};
```

Why subscribe first? There's a race condition: if you call `getSession()` first, then subscribe, you might miss the auth event that fires while `getSession()` is pending. Subscribing first ensures nothing is missed.

What the hook provides:

- `session` — the full Supabase session object (contains JWT token, user info)
- `user` — shortcut to `session?.user`
- `loading` — `true` while the session is being restored from storage on page load
- `signOut()` — logs the user out

8.1 The `RequireAuth` Guard

```
const RequireAuth = ({ children }) => {
  const { session, loading } = useAuth();
  const location = useLocation();

  if (loading) return null; // wait for auth to resolve, show nothing
  if (!session) return <Navigate to="/auth" replace state={{ from: location }} />;
  return <OnboardingGate>{children}</OnboardingGate>;
};
```

This is a wrapper that wraps all protected routes. If the user isn't logged in, they're redirected to `/auth`. The `state={{ from: location }}` saves where they were trying to go, so after login they're sent back there.

`loading` is critical — without it, authenticated users would be briefly redirected to `/auth` every time they load the app, because the session starts as `null` and takes a moment to restore from storage.

9. The Brain of the App — `useAppState`

File: `src/hooks/useAppState.tsx`

This is the most complex file in the codebase. It manages:

- All habits, user profile, and calendar events
- Loading data from `localStorage` and Supabase
- Syncing changes back to Supabase
- The daily login bonus system
- The weekly recap system
- The streak warning notification
- All mutation actions (toggle habit, freeze streak, purchase item, etc.)

9.1 Architecture Overview

```

localStorage (fast, offline)
  ↓
  useAppState (the "server")
  ↓
  Supabase (cloud, persistent)

```

On load: data is read from `localStorage` instantly, then Supabase is queried in the background. When Supabase responds, the local state is **merged** (not replaced) with the cloud data — the better value wins for each field.

On any change: the state is updated immediately (optimistic update), `localStorage` is written synchronously, and Supabase is called asynchronously in the background.

9.2 Important Constants

```

export const FREE_HABIT_LIMIT = 5; // free users can have 5 permanent habits
export const LOGIN_BONUSES = [5, 10, 15, 20, 25, 30, 50];
// Day 1 = 5 coins, Day 2 = 10, ..., Day 7 = 50 coins

```

9.3 The Context Interface (Ctx)

```

interface Ctx {
  // Data
  habits: Habit[];
  user: UserState;
  events: CalendarEvent[];
  syncing: boolean; // true while loading from Supabase

  // Habit actions
  addHabit: (payload) => void;
  updateHabit: (id, payload) => void;
  deleteHabit: (id) => void;
  addTempHabit: (payload) => void; // ad-based today-only habit
  toggleCompletion: (id, date) => void;

  // User actions
  freezeStreak: () => void; // spend 50 coins to freeze today
  recoverStreak: (id, date, via) => void; // recover missed habit
  purchaseItem: (item) => void; // buy shop item
  upgradeToPro: () => void;
  cancelPro: () => void;
  watchAd: () => number; // simulate watching ad, returns coins earned
  setReminder: (enabled, time) => void;
  setTheme: (theme) => void;

  // Events

```

```

addEvent: (event) => void;
deleteEvent: (id) => void;

// UI state
loginBonus: { coins, day } | null;
clearLoginBonus: () => void;
weeklyRecap: WeeklyRecap | null;
clearWeeklyRecap: () => void;
unlockEvent: UnlockEvent | null;
clearUnlockEvent: () => void;
}

```

9.4 State Initialization

```

const [habits, setHabits] = useState<Habit[]>(() => loadHabits());
const [user, setUser] = useState<UserState>(() => loadUser());
const [events, setEvents] = useState<CalendarEvent[]>(() => loadEvents());

```

The `() => loadHabits()` syntax is a **lazy initializer** — it runs `loadHabits()` only once when the component first mounts, not on every re-render. This is important because `JSON.parse` is slow.

9.5 Refs — Fighting Stale Closures

```

const habitsRef = useRef(habits);
const userRef = useRef(user);

useEffect(() => { habitsRef.current = habits; }, [habits]);
useEffect(() => { userRef.current = user; }, [user]);

```

This is one of the trickiest React patterns. Here's the problem:

When you create a function with `useCallback`, it "captures" the values of variables at the time it was created. If `coins` was 100 when `toggleCompletion` was created, it will forever see `coins = 100` even if the user later earned more coins — this is a **stale closure**.

The solution: keep a `ref` that always points to the latest value. Refs don't trigger re-renders and always reflect the current value. Functions that need "read the latest state" use `userRef.current` instead of the `user` state variable directly.

9.6 The Cloud Load Effect

This is the most important `useEffect` in the app:

```

useEffect(() => {
  if (authLoading || !authUser) {
    if (!authLoading && !authUser) setSyncing(false);
  }
}, [authLoading, authUser]);

```

```

    return;
  }

  const load = async () => {
    // 1. Load habits from cloud
    const { data: cloudHabits } = await supabase
      .from("habits")
      .select("*")
      .eq("user_id", authUser.id);

    // 2. Merge cloud habits with local habits
    // (union of completions – if either has a completion, keep it)
    setHabits(local => {
      const localMap = new Map(local.map(h => [h.id, h]));
      const merged = (cloudHabits ?? []).map(ch => {
        const lh = localMap.get(ch.id);
        if (!lh) return ch;
        // Keep all completions from both sources
        const allCompletions = [...new Set([...ch.completions,
...lh.completions])].sort();
        return { ...ch, completions: allCompletions };
      });
      // Keep local-only habits (not yet synced)
      const cloudIds = new Set(merged.map(h => h.id));
      const localOnly = local.filter(h => !cloudIds.has(h.id));
      return [...merged, ...localOnly];
    });

    // 3. Load profile from cloud
    const { data: profile, error: profileErr } = await supabase
      .from("profiles")
      .select("*")
      .eq("user_id", authUser.id)
      .maybeSingle();

    if (profile) {
      // Merge: take the better value for accumulating fields
      setUser(local => ({
        ...local,
        coins: Math.max(local.coins, profile.coins),
        unlocked: [...new Set([...local.unlocked, ...profile.unlocked])],
        loginStreak: Math.max(local.loginStreak ?? 0, profile.login_streak ?? 0),
        // ... etc
      }));
    } else if (profileErr?.code === "PGRST116") {
      // "no rows" – new user, create the profile
      await supabase.from("profiles").insert({ user_id: authUser.id, ...defaults
    });
  }

  setSyncing(false);
};

```



```
load();
}, [authUser?.id, authLoading]);
```

Why merge instead of replace? The user might have added habits or completed them while offline. Simply overwriting with cloud data would lose those changes. The merge strategy takes the union of both datasets.

9.7 Push Functions

Every write to Supabase is wrapped in a small `useCallback` function:

```
const pushHabit = useCallback(async (habit: Habit) => {
  if (!authUser) return;
  const { error } = await supabase.from("habits").upsert({
    id: habit.id,
    user_id: authUser.id,
    name: habit.name,
    // ... all fields
  });
  if (error) { console.error(error); warnSyncFailed(); }
}, [authUser, warnSyncFailed]);
```

`useCallback` memoizes the function — it won't be recreated on every render unless `authUser` or `warnSyncFailed` changes.

9.8 Toggle Completion

```
const toggleCompletion = useCallback((habitId, date) => {
  const key = toDateKey(date);
  let toSync: Habit | undefined;

  setHabits(prev => prev.map(h => {
    if (h.id !== habitId) return h; // not this habit, unchanged
    const completions = h.completions.includes(key)
      ? h.completions.filter(c => c !== key) // un-complete: remove date
      : [...h.completions, key].sort(); // complete: add date
    toSync = { ...h, completions };
    return toSync!;
  }));

  // Also check if the user unlocked a milestone (100 completions, etc.)
  // ... milestone celebration logic

  // Save locally and push to cloud
  if (toSync) {
    const updatedHabits = habitsRef.current.map(h => h.id === habitId ? toSync! : h);
    saveHabits(updatedHabits);
    pushHabit(toSync);
  }
});
```

```
// Update profile stats
const newUser = { ...userRef.current, coins: userRef.current.coins + 5 };
setUser(newUser);
pushProfile(newUser);
}
}, [pushHabit, pushProfile]);
```

Optimistic update pattern: The state is updated instantly (the UI checkbox responds immediately), and the Supabase write happens in the background. If the write fails, `warnSyncFailed()` shows a toast but the local state is still correct.

9.9 The `warnSyncFailed` Pattern

```
const syncWarnedRef = useRef(false);

const warnSyncFailed = useCallback(() => {
  if (syncWarnedRef.current) return; // only show once per session
  syncWarnedRef.current = true;
  toast.error("Sync failed", {
    description: "Your data is saved locally but couldn't reach the cloud."
  });
}, []);
```

9.10 Daily Login Bonus

```
useEffect(() => {
  if (syncing || !authUser || loginBonusChecked.current) return;
  loginBonusChecked.current = true; // prevents running again this session

  const today = toDateKey(new Date());
  const current = userRef.current;
  if (current.lastLoginDate === today) return; // already claimed today

  const yesterday = toDateKey(new Date(Date.now() - 86_400_000));
  const newStreak = current.lastLoginDate === yesterday
    ? Math.min((current.loginStreak ?? 0) + 1, 7) // continue streak (cap at 7)
    : 1; // streak broken or first login

  const bonusCoins = LOGIN_BONUSES[newStreak - 1]; // LOGIN_BONUSES[0] = 5, etc.
  setLoginBonus({ coins: bonusCoins, day: newStreak });
  // ... update user state and push to cloud
}, [syncing, authUser?.id]);
```

9.11 Weekly Recap

```

useEffect(() => {
  if (syncing || !authUser) return;
  const today = new Date();
  if (today.getDay() !== 6) return; // 6 = Saturday

  const satKey = toDateKey(today);
  if (localStorage.getItem("lastRecapDate") === satKey) return; // already shown

  // Calculate this week vs last week
  const currentHabits = habitsRef.current;
  // ... calculate rate7, prevRate, streak, totalDone
  // ... compute weekLabel

  setWeeklyRecap({ rate, prevRate, streak, totalDone, weekLabel });
  localStorage.setItem("lastRecapDate", satKey);
}, [syncing, authUser?.id]);

```

9.12 Streak Warning at 8 PM

```

useEffect(() => {
  if (!authUser) return;

  const MESSAGES = [
    "Sprout is stress-eating and it's YOUR fault 🌱🤖...",
    "Your streak has entered its villain arc...",
    // ... 10 total messages
  ];

  const check = () => {
    if (Notification.permission !== "granted") return;
    const now = new Date();
    if (now.getHours() < 20) return; // before 8 PM

    const today = toDateKey(now);
    if (localStorage.getItem("lastStreakWarning") === today) return;

    // Check if any scheduled habits are incomplete
    const incomplete = habitsRef.current.filter(
      h => isHabitScheduled(h, now) && !h.completions.includes(today)
    );
    if (incomplete.length === 0) return;

    // Pick a random funny message and fire the notification
    const msg = MESSAGES[Math.floor(Math.random() * MESSAGES.length)];
    new Notification("⚠️ Streak at risk!", { body: msg });
    localStorage.setItem("lastStreakWarning", today);
  };

  check(); // check immediately on mount
  const id = window.setInterval(check, 60_000); // then every minute

```

```
return () => window.clearInterval(id); // cleanup on unmount
}, [authUser?.id]);
```

10. Habit Logic Library (`lib/habits.ts`)

File: `src/lib/habits.ts`

This file contains all the pure functions that compute habit statistics. "Pure" means they take inputs and return outputs without any side effects — no state changes, no API calls.

10.1 `toDateKey(date)` — The Core Date Format

```
export const toDateKey = (d: Date): string => {
  const y = d.getFullYear();
  const m = String(d.getMonth() + 1).padStart(2, "0"); // months are 0-indexed!
  const day = String(d.getDate()).padStart(2, "0");
  return `${y}-${m}-${day}`; // "2024-01-15"
};
```

All dates in the app are stored as "YYYY-MM-DD" strings. This is important:

- It's timezone-safe (no UTC conversion issues)
- It sorts alphabetically AND chronologically
- It's easy to compare: `"2024-01-15" > "2024-01-14"` is `true`

10.2 `isHabitScheduled(habit, date)` — Should This Habit Appear Today?

```
export const isHabitScheduled = (habit: Habit, date: Date): boolean =>
  habit.days.includes(date.getDay() as WeekDay);
```

`date.getDay()` returns 0 (Sunday) through 6 (Saturday). If that number is in the habit's `days` array, the habit is scheduled for that day.

10.3 `getCurrentStreak(habit, freezes)` — Per-Habit Streak

```
export const getCurrentStreak = (habit: Habit, freezes: string[] = []): number =>
{
  if (habit.days.length === 0) return 0;
  const frozenSet = new Set(freezes); // Set is faster for lookups than array
  let streak = 0;
  const cursor = new Date();

  for (let i = 0; i < 365; i++) {
    const key = toDateKey(cursor);
    if (isHabitScheduled(habit, cursor)) {
```

```

    if (isCompletedOn(habit, cursor) || frozenSet.has(key)) {
      streak++; // completed or frozen = streak continues
    } else {
      if (i === 0) { // today, not yet done – give grace, skip
        cursor.setDate(cursor.getDate() - 1);
        continue;
      }
      break; // missed a past day – streak ends
    }
  }
  cursor.setDate(cursor.getDate() - 1); // go back one day
}
return streak;
};

```

The algorithm walks backwards from today: if today's habit isn't done yet, it skips (you might still do it). For every past scheduled day, it must be either completed or frozen. The first missed past day breaks the streak.

10.4 `getDailyStreak(habits, freezes)` — The "Showed Up" Streak

This is the streak shown prominently in the UI — it counts consecutive days where **any** habit was completed (not a per-habit streak):

```

export const getDailyStreak = (habits: Habit[], freezes: string[] = []): number =>
{
  if (habits.length === 0) return 0;
  const frozenSet = new Set(freezes);
  let streak = 0;
  const cursor = new Date();

  for (let i = 0; i < 365; i++) {
    const key = toDateKey(cursor);
    const scheduled = habits.filter(h => isHabitScheduled(h, cursor));

    if (scheduled.length === 0) {
      // No habits scheduled this day – skip it entirely (rest day)
      cursor.setDate(cursor.getDate() - 1);
      continue;
    }

    const anyCompleted = scheduled.some(h => h.completions.includes(key));

    if (anyCompleted || frozenSet.has(key)) {
      streak++;
    } else if (i === 0) {
      // Today not done yet – don't break the streak
    } else {
      break; // scheduled day with nothing done
    }
    cursor.setDate(cursor.getDate() - 1);
  }
}

```

```
    return streak;
  };

```

10.5 `completionRate(habits, days)` — Percentage Completion

```
export const completionRate = (habits: Habit[], days: number): number => {
  let scheduled = 0, completed = 0;
  const today = new Date();

  for (let i = 0; i < days; i++) {
    const d = new Date(today);
    d.setDate(today.getDate() - i); // go back i days
    for (const h of habits) {
      if (isHabitScheduled(h, d)) {
        scheduled++;
        if (isCompletedOn(h, d)) completed++;
      }
    }
  }
  return scheduled === 0 ? 0 : Math.round((completed / scheduled) * 100);
};

```

For example, if you have 3 habits each scheduled 7 days, `scheduled = 21`. If you completed 15 of them, the rate is `Math.round(15/21 * 100) = 71%`.

10.6 `getStreakRecoveryDate(habit, freezes)` — Is a Streak Recoverable?

```
export const getStreakRecoveryDate = (habit: Habit, freezes: string[] = []):
string | null => {
  if (habit.days.length === 0) return null;
  const yesterday = new Date();
  yesterday.setDate(yesterday.getDate() - 1);
  const yKey = toDateKey(yesterday);

  if (!isHabitScheduled(habit, yesterday)) return null; // not scheduled
  yesterday
  if (habit.completions.includes(yKey)) return null; // already done
  if (freezes.includes(yKey)) return null; // already frozen

  // Only offer recovery if there was recent activity (not abandoning the habit)
  for (let i = 2; i <= 7; i++) {
    const d = new Date();
    d.setDate(d.getDate() - i);
    const key = toDateKey(d);
    if (habit.completions.includes(key) || freezes.includes(key)) return yKey;
  }
  return null; // no recent activity, don't offer recovery
};

```

11. Badges Library (`lib/badges.ts`)

File: `src/lib/badges.ts`

Badges are computed on-the-fly from current data — they're never stored, just calculated each render. This means they automatically disappear if conditions are no longer met (though in practice that shouldn't happen with these specific conditions).

```
export const ALL_BADGES: Badge[] = [
  { id: "first_step", name: "First Step", emoji: "🌱", description:
    "Complete your first habit" },
  { id: "week_warrior", name: "Week Warrior", emoji: "🔥", description: "7-day
    streak on any habit" },
  { id: "month_master", name: "Month Master", emoji: "🏆", description: "30-day
    streak on any habit" },
  { id: "streak_legend", name: "Streak Legend", emoji: "🦊", description: "60-day
    streak" },
  { id: "century_club", name: "Century Club", emoji: "👑", description: "100-
    day streak" },
  { id: "habit_builder", name: "Habit Builder", emoji: "💪", description: "Create
    5+ habits" },
  { id: "coin_hoarder", name: "Coin Hoarder", emoji: "💰", description: "500+
    coins" },
  { id: "collector", name: "Collector", emoji: "🎮", description: "Unlock
    5 shop items" },
  { id: "loyal_plant", name: "Loyal Plant", emoji: "🌿", description: "7-day
    login streak" },
  { id: "overachiever", name: "Overachiever", emoji: "⚡", description: "100
    total completions" },
];

export const computeEarnedBadges = (habits: Habit[], user: UserState): Badge[] =>
{
  const earned = new Set<string>();

  const totalCompletions = habits.reduce((sum, h) => sum + h.completions.length,
0);
  const maxStreak = Math.max(
    ...habits.map(h => getLongestStreak(h)),
    ...habits.map(h => getCurrentStreak(h, user.streakFreezes ?? []))
  );

  if (totalCompletions >= 1) earned.add("first_step");
  if (maxStreak >= 7) earned.add("week_warrior");
  if (maxStreak >= 30) earned.add("month_master");
  if (maxStreak >= 60) earned.add("streak_legend");
  if (maxStreak >= 100) earned.add("century_club");
  if (habits.length >= 5) earned.add("habit_builder");
  if (user.coins >= 500) earned.add("coin_hoarder");
  if (user.unlocked?.length >= 5) earned.add("collector");
  if ((user.loginStreak ?? 0) >= 7) earned.add("loyal_plant");
}
```

```
if (totalCompletions >= 100)      earned.add("overachiever");

return ALL_BADGES.filter(b => earned.has(b.id));
};
```

12. Shop Items Library (`lib/shopItems.ts`)

File: `src/lib/shopItems.ts`

A shared catalog of purchasable items, imported by both the full Shop page and the home-screen shop widget:

```
export const SHOP_ITEMS = [
  { id: "default",    name: "Sprout",    emoji: "🌱", cost: 0,   kind: "Skin"
},
  { id: "hat-leaf",   name: "Leaf hat",  emoji: "🍃", cost: 30,  kind: "Hat"
},
  { id: "hat-party",  name: "Party hat",  emoji: "🎉", cost: 45,  kind: "Hat"
},
  { id: "bg-meadow",  name: "Meadow",    emoji: "🌸", cost: 60,  kind: "Theme"
},
  { id: "pet-bunny",  name: "Bunny pal",  emoji: "🐰", cost: 80,  kind: "Pet"
},
  { id: "hat-crown",  name: "Royal crown", emoji: "👑", cost: 150, kind: "Hat"
},
  { id: "bg-space",   name: "Cosmic",    emoji: "🌌", cost: 200, kind: "Theme"
},
  { id: "pet-dragon", name: "Tiny dragon", emoji: "🐉", cost: 250, kind: "Pet"
},
] as const;
```

`as const` tells TypeScript to treat every value as a literal type (e.g. `cost` is exactly `30`, not just `number`). This enables stronger type checking.

13. App Entry Point & Routing

File: `src/App.tsx`

This file is the root of the application. It:

1. Sets up global providers (things that wrap the entire app)
2. Defines all URL routes
3. Handles authentication protection

13.1 Provider Stack

<code><QueryClientProvider client={queryClient}></code>	← React Query (data fetching cache)
<code><TooltipProvider></code>	← Tooltips
<code><Toaster /> <Sonner /></code>	← Toast notifications
<code><AuthProvider></code>	← Authentication state
<code><AppStateProvider></code>	← All app data
<code><ThemeProvider></code>	← Dark/light mode
<code><AppShell /></code>	
<code></ThemeProvider></code>	
<code></AppStateProvider></code>	
<code></AuthProvider></code>	
<code></TooltipProvider></code>	
<code></QueryClientProvider></code>	

Each **Provider** makes its context available to everything inside it. Think of it as nested scopes.

13.2 Routes

```

<Routes>
  <Route path="/auth" element={<AuthPage />} />      ← public
  <Route path="*" element={
    <RequireAuth>                                     ← protected
      <AppLayout>
        <Routes>
          <Route path="/" element={<Index />} />
          <Route path="/calendar" element={<CalendarPage />} />
          <Route path="/friends" element={<FriendsPage />} />
          <Route path="/character" element={<CharacterPage />} />
          <Route path="/ai" element={<AIPage />} />
          <Route path="/profile" element={<ProfilePage />} />
          <Route path="/upgrade" element={<UpgradePage />} />
          <Route path="/coach" element={<CoachPage />} />
          <Route path="/notifications" element={<NotificationsPage />} />
        </Routes>
      </AppLayout>
    </RequireAuth>
  } />
</Routes>

```

Only `/auth` is public. Every other route is wrapped in `RequireAuth`, which redirects to `/auth` if not logged in.

13.3 The Splash Screen

```

const AppShell = () => {
  const [splashDone, setSplashDone] = useState(false);

  useEffect(() => {
    // Skip splash if seen this browser session (page refresh doesn't re-show it)
    if (sessionStorage.getItem("splash_seen") === "1") setSplashDone(true);
  });
}

```

```

    }, []);

    return (
      <>
        {!splashDone && <SplashScreen onDone={finishSplash} />}
        <BrowserRouter>...</BrowserRouter>
      </>
    );
  };
};

```

`sessionStorage` is like `localStorage` but clears when the tab is closed. The splash screen only shows once per browser session.

13.4 The Onboarding Gate

```

const OnboardingGate = ({ children }) => {
  const { user, syncing } = useApp();

  if (syncing) return <LoadingSpinner />; // wait for data
  if (!user.onboardingDone) return <OnboardingFlow />; // first time
  return children; // normal flow
};

```

This sits between `RequireAuth` and the actual pages. If the user hasn't completed onboarding (`onboardingDone = false`), they see the setup wizard instead of whatever page they were trying to visit.

14. Layout & Navigation

File: `src/components/AppLayout.tsx`

This component wraps all protected pages with the bottom navigation bar:

```

const tabs = [
  { to: "/", label: "Home", icon: Home, end: true },
  { to: "/calendar", label: "Calendar", icon: CalIcon },
  { to: "/friends", label: "Friends", icon: Users },
  { to: "/character", label: "Shop", icon: ShoppingBag },
  { to: "/ai", label: "AI", icon: Bot },
  { to: "/profile", label: "Profile", icon: User },
];

```

The `end: true` on the Home route means it only highlights as "active" when the URL is exactly `/`, not on `/calendar` or other paths that start with `/`.

The `NavLink` component from React Router automatically adds active styling. The `animate-slide-up` class on the content area creates a subtle slide-in animation each time you navigate to a new page (because `key=`

`{loc.pathname}` causes React to remount the div on path changes).

15. Page: Home (`Index.tsx`)

File: `src/pages/Index.tsx`

The home page is the most feature-rich page. It contains many sections rendered from top to bottom:

15.1 Sections (in order)

1. **Login Bonus Modal** — shown as a full-screen overlay if `loginBonus` is set
2. **Weekly Recap Modal** — shown as a full-screen overlay on Saturdays
3. **Header** — greeting, day name, coin balance, sync spinner
4. **Hero Card** — mascot, `Sprout` `says` message, 7-day rate, today's progress, streak, freeze button
5. **Badges** — earned badges scrolled horizontally
6. **Streak Recovery** — `StreakRecoveryCard` for recoverable missed habits
7. **Expired Temp Habits** — habits created via ads that have expired
8. **Today's Habits** — habits scheduled for today
9. **All Habits** — active habits not scheduled today
10. **Stats Section** — full analytics (4 stat cards, weekly trend chart, heatmap, per-habit streaks, insight)
11. **Friends Widget** — invite code + "View league" link
12. **Shop Widget** — next 3 purchasable items

15.2 Key Calculations

```
const activeHabits = habits.filter(isHabitActive); // excludes expired temp
habits
const todays = habitsForDay(activeHabits, today); // scheduled for today
const doneToday = todays.filter(h => h.completions.includes(todayKey)).length;
const dailyStreak = getDailyStreak(activeHabits, user.streakFreezes ?? []);
const rate7 = completionRate(activeHabits, 7);
```

15.3 The WeeklyTrend SVG

The bar chart on the home page is drawn as raw SVG (no chart library). This keeps bundle size small:

```
const WeeklyTrend = ({ rates }) => {
  const W = 280, H = 90; // viewBox dimensions
  const padX = 8, chartH = 58;
  const step = (W - padX * 2) / n; // width of each bar section

  return (
    <svg viewBox={`0 0 ${W} ${H}`}>
      {rates.map((r, i) => {
        const barH = (r.rate / 100) * chartH; // bar height proportional to %
        const barY = padT + chartH - barH; // y position (SVG y goes down)
        // ... draws rect and text
      })}
```

```

    }}}
  </svg>
);
};

```

16. Page: Calendar ([CalendarPage.tsx](#))

File: [src/pages/CalendarPage.tsx](#)

The calendar page has two responsibilities: showing a month grid and showing the selected day's details.

16.1 Building the Calendar Grid

```

const firstDay = new Date(year, month, 1);
const startWeekday = firstDay.getDay(); // which column the 1st falls on
const daysInMonth = new Date(year, month + 1, 0).getDate(); // days in this month

const cells: (Date | null)[] = [];
for (let i = 0; i < startWeekday; i++) cells.push(null); // empty cells before
day 1
for (let d = 1; d <= daysInMonth; d++) cells.push(new Date(year, month, d));
while (cells.length % 7 !== 0) cells.push(null); // fill last row

```

`new Date(year, month + 1, 0)` is a trick: "day 0 of next month" = "last day of this month".

16.2 Day Status Dots

Each calendar cell shows a tiny colored dot:

- Green dot = all scheduled habits completed
- Yellow dot = some completed
- Red dot = none completed (past days only)
- Blue dot = has a calendar event

```

const dayStat = (d: Date) => {
  const sched = habitsForDay(habits, d);
  if (sched.length === 0) return { state: "none" };
  const done = sched.filter(h => isCompletedOn(h, d)).length;
  if (done === 0) return { state: "missed" };
  if (done === sched.length) return { state: "all" };
  return { state: "partial" };
};

```

17. Page: Friends ([FriendsPage.tsx](#))

File: `src/pages/FriendsPage.tsx`

The friends page has two tabs: **League** (leaderboard) and **Challenges**.

17.1 League Tab

Shows a ranked leaderboard of accepted friends, sorted by their `best_streak` from the `profiles` table. Users search by friend code to add new friends.

```
const addFriend = async () => {
  // Find user by friend_code
  const { data: target } = await supabase
    .from("profiles")
    .select("user_id")
    .eq("friend_code", code.trim().toUpperCase())
    .maybeSingle();

  // Insert friendship record
  await supabase.from("friendships").insert({
    requester_id: authUser.id,
    addressee_id: target.user_id,
    status: "accepted", // auto-accept for simplicity
  });
};
```

17.2 Challenges Tab

Group challenges with:

- A 6-character invite code (generated as `Math.random().toString(36).slice(2, 8).toUpperCase()`)
- Daily "Mark done today" tracking
- A leaderboard sorted by days completed
- Duration: 7, 14, 21, or 30 days

Since the `challenges` and `challenge_members` tables aren't in the auto-generated Supabase types, the code uses a type cast:

```
const db = supabase as any; // escape type checking for new tables
await db.from("challenges").insert({ ... });
```

18. Page: Shop / Character (`CharacterPage.tsx`)

File: `src/pages/CharacterPage.tsx`

The shop page shows:

- The user's mascot character
- A "next unlock" progress bar
- A "Watch Ad" button (simulates 3 seconds, earns 5–50 coins)
- A 3-column grid of all shop items

```
const next = SHOP_ITEMS.filter(i => !user.unlocked.includes(i.id))[0];
const progress = next ? Math.min(100, Math.round((user.coins / next.cost) * 100))
: 100;
```

The "next item" is the first item in `SHOP_ITEMS` that the user doesn't own yet. Progress shows how close they are to affording it.

Purchasing

```
onClick={() => !owned && purchaseItem(it)}
disabled={owned || !canAfford}
```

The `purchaseItem` function in `useAppState`:

1. Deducts `item.cost` from `user.coins`
2. Adds `item.id` to `user.unlocked`
3. Triggers the `CelebrationOverlay` animation
4. Saves to `localStorage` and pushes to Supabase

19. Page: AI (`AIPage.tsx`)

File: `src/pages/AIPage.tsx`

The AI page has two tabs: **Chat** and **Coach**. Both are Pro-only features.

19.1 Chat Tab

A standard chat interface with persistent history:

```
// History saved to localStorage, keyed per user
const chatKey = (uid: string) => `sprout_chat_${uid}`;
const loadHistory = (uid: string): ChatMessage[] => {
  // JSON.parse from localStorage
};
const saveHistory = (uid: string, msgs: ChatMessage[]) =>
  localStorage.setItem(chatKey(uid), JSON.stringify(msgs.slice(-60))); // keep
  last 60
```

Suggestion chips appear when the chat only has the welcome message:

```

{messages.length <= 1 && !loading && (
  <div className="flex flex-wrap gap-2">
    [{"🏳️🌈 How am I doing overall?", "😓 Why do I keep missing habits?",
    ...].map(chip => (
      <button onClick={() => send(chip)}>
        {chip}
      </button>
    ))}
  </div>
)}

```

Sending a message:

```

const send = async (override?: string) => {
  const text = (override ?? input).trim(); // use chip text or typed text
  setMessages(prev => [...prev, { role: "user", content: text }]);
  setLoading(true);

  const { data, error } = await supabase.functions.invoke("ai-chat", {
    body: {
      message: text,
      history: messages.slice(-10), // last 10 messages for context
      habits: summarizeHabits(habits), // habit data for personalization
      userContext: { displayName, goal, coins, characterName },
    },
  });

  setMessages(prev => [...prev, { role: "assistant", content: data.reply }]);
};

```

19.2 Coach Tab

Uses the `CoachPanel` component (see Section 26).

20. Page: Profile (`ProfilePage.tsx`)

File: `src/pages/ProfilePage.tsx`

Settings and habit management in one page:

- **User card** — name, Pro badge, habit count, coin balance
- **Friend code** — with copy button
- **Character shop shortcut** — link to `/character`
- **Pro banner** — upgrade prompt for free users, or "Open AI Coach" for Pro users
- **All habits list** — every habit with day indicators and delete button
- **Settings section** — theme picker (light/dark/system), daily reminder toggle + time picker, subscription link, notifications link, sign out

Theme Picker

```
const themeOptions = [
  { value: "light" as const, label: "Light", icon: Sun },
  { value: "dark" as const, label: "Dark", icon: Moon },
  { value: "system" as const, label: "System", icon: Monitor },
];

// "as const" narrows the type so TypeScript knows the exact string value
```

Reminder Toggle

```
<Switch
  checked={reminderEnabled}
  onCheckedChange={(v) => setReminder(v, reminderTime)}
/>
```

`setReminder` in `useAppState` requests browser notification permission if not already granted, updates the user state, and schedules a polling loop that checks at the configured time whether there are incomplete habits.

21. Page: Upgrade (`UpgradePage.tsx`)

File: `src/pages/UpgradePage.tsx`

The Pro subscription paywall page. Key feature: the **"Your streak would be X days with Pro"** card.

The Pro Streak Calculation

```
const proStreak = (() => {
  if (habits.length === 0) return 0;

  // Find the earliest habit creation date (before this, the user had no habits)
  const earliest = habits
    .reduce((min, h) => h.createdAt < min ? h.createdAt : min,
  habits[0].createdAt)
    .slice(0, 10);

  let streak = 0;
  const cursor = new Date();

  for (let i = 0; i < 365; i++) {
    const key = toDateKey(cursor);
    if (key < earliest) break; // before habits existed

    const scheduled = habits.filter(h => isHabitScheduled(h, cursor));
```



```

    if (scheduled.length === 0) {
      cursor.setDate(cursor.getDate() - 1);
      continue; // no habits scheduled – rest day, skip
    }

    // With Pro: every scheduled day counts (auto-freeze covers missed days)
    // The only exception: today, if nothing's been done yet
    if (i === 0 && !scheduled.some(h => h.completions.includes(key))) {
      cursor.setDate(cursor.getDate() - 1);
      continue;
    }

    streak++;
    cursor.setDate(cursor.getDate() - 1);
  }
  return streak;
})();

const lostDays = proStreak - actualStreak;

```

This shows the user exactly how many days they "lost" to missed streaks — a direct loss-aversion message.

22. Page: Auth ([AuthPage.tsx](#))

File: [src/pages/AuthPage.tsx](#)

The sign-in/sign-up page using Supabase Auth:

```

await supabase.auth.signInWithPassword({ email, password });
// or
await supabase.auth.signUp({ email, password });

```

After a successful login, Supabase fires an auth state change event. The [AuthProvider](#) in [useAuth.tsx](#) catches this and updates [session](#), which causes [RequireAuth](#) to re-render and navigate to the intended destination.

23. Page: Notifications ([NotificationsPage.tsx](#))

File: [src/pages/NotificationsPage.tsx](#)

Manages notification preferences stored in the [notification_preferences](#) Supabase table:

- Daily reminder (enabled + time)
- Streak warnings (enabled)
- Milestone alerts (enabled)
- Weekly summary (enabled + day of week)

These are separate from the [userState.reminders](#) settings and allow more granular control.

24. Component: HabitCard

File: `src/components/HabitCard.tsx`

Each habit in the list is rendered by this component:

```
export const HabitCard = ({ habit, date }: { habit: Habit; date: Date }) => {
  const { toggleCompletion, user } = useApp();
  const done = isCompletedOn(habit, date);
  const streak = getCurrentStreak(habit, user.streakFreezes ?? []);
  const isTemp = !!habit.expiresAt; // "!!" converts to boolean
  // ...
};
```

Key behavior:

- Tapping either the emoji icon OR the check circle calls `toggleCompletion`
- The pencil icon opens a `CreateHabitDialog` in edit mode
- A flame icon shows the habit's current streak
- Temporary habits show a "Today only" pill badge
- When completed, the background changes to `bg-secondary` and text gets a strikethrough

The `colorMap`

```
const colorMap: Record<string, string> = {
  primary: "bg-primary text-primary-foreground",
  accent: "bg-accent text-accent-foreground",
  success: "bg-success text-success-foreground",
  warning: "bg-warning text-warning-foreground",
};
```

Habit colors are stored as semantic names ("`primary`", "`accent`") not hex values. This way they automatically adapt to light/dark themes.

25. Component: CreateHabitDialog

File: `src/components/CreateHabitDialog.tsx`

This component serves dual purposes — creating new habits and editing existing ones. It detects mode via `initialHabit`:

```
const isEdit = !!initialHabit;

const submit = () => {
  const payload = { name, emoji, color, days, reminderTime: reminderTime ||
```

```

undefined };
if (isEdit) {
  updateHabit(initialHabit!.id, payload); // "!" = "I know this isn't null"
} else {
  addHabit(payload);
}
};

```

Free Habit Limit Gate

```

const permanentCount = habits.filter(h => !h.expiresAt).length;
const atLimit = !user.isPro && permanentCount >= FREE_HABIT_LIMIT && !isEdit;

```

When `atLimit` is true, instead of showing the normal form, it shows:

- A message explaining the limit (5 habits)
- A "Watch Ad — add for today only" button (creates a temp habit)
- An "Upgrade to Pro — unlimited habits" button

Controlled vs. Uncontrolled Mode

The dialog can work in two modes:

```

// Uncontrolled: manages its own open/close state
<CreateHabitDialog trigger={<Button>Add habit</Button>} />

// Controlled: parent manages the open/close state
<CreateHabitDialog
  initialHabit={habit}
  open={editOpen}
  onOpenChange={setEditOpen}
/>

```

The controlled mode is used by `HabitCard` for editing — the parent manages when the dialog opens/closes.

26. Component: CoachPanel

File: `src/components/CoachPanel1.tsx`

The AI coach panel is used in two places: the `AIPage` Coach tab and the standalone `CoachPage`. It's extracted as a reusable component to avoid code duplication.

Section 1: Today's Snapshot

```

const todaysHabits = habits.filter(h => isHabitScheduled(h, today));
const doneToday = todaysHabits.filter(h => h.completions.includes(todayKey));
const currentStreak = getDailyStreak(habits, user.streakFreezes ?? []);

// Week-over-week calculation
let sched7 = 0, done7 = 0, schedPrev = 0, donePrev = 0;
for (let i = 0; i < 14; i++) {
  // days 0-6 = this week, days 7-13 = last week
  if (i < 7) { sched7++; ... } else { schedPrev++; ... }
}
const trendDiff = rate7 - ratePrev; // positive = improving

```

The "vs last week" number in the snapshot is color-coded: green for improvement, red for decline.

Section 2: Pattern Insights

```

// 14-day per-habit stats
const habitStats = habits.map(h => {
  let sched = 0, done = 0;
  for (let i = 0; i < 14; i++) {
    const d = new Date(today); d.setDate(today.getDate() - i);
    if (isHabitScheduled(h, d)) { sched++; if (isCompletedOn(h, d)) done++; }
  }
  return { h, rate: sched === 0 ? null : done / sched, sched };
}).filter(s => s.sched >= 2); // must have at least 2 scheduled days

// Sort by completion rate to find best and worst
const sortedStats = [...habitStats].sort((a, b) => (b.rate ?? 0) - (a.rate ?? 0));
const starHabit = sortedStats[0];
const struggleHabit = sortedStats[sortedStats.length - 1];

// Weekday patterns over 28 days
const dayData = [0, 1, 2, 3, 4, 5, 6].map(wd => {
  // ... count completions on this weekday over the past 4 weeks
});

```

Progress bars for each habit use inline `style={{ width: "73%" }}` — this is the one case where inline styles are used instead of Tailwind, because the width is dynamic.

Section 3: AI Coaching Plan

Instead of calling the generic `ai-coach` edge function, the coach panel calls `ai-chat` with a carefully crafted prompt that includes all the locally computed patterns:

```

const message = `You are a direct, no-fluff habit coach. My real habit data:
${patterns}. My goal: ${user.goal}. Give me exactly 3 things: (1) One specific fix
for my weakest area with a concrete action step, (2) One thing to protect what is

```

already working, (3) One thing to do TODAY. Keep each point to 2 sentences max. Be specific to my data, not generic.`;

By injecting the real pattern data into the prompt (best habit name + %, worst habit + %, best weekday + %, trend), the AI response is forced to be specific rather than generic.

27. Component: StreakRecoveryCard

File: `src/components/StreakRecoveryCard.tsx`

Shows a "Streak at risk" card for habits that:

- Were scheduled yesterday
- Were NOT completed yesterday
- Had activity in the 2–7 days before yesterday (proving it's not an abandoned habit)

```
export const StreakRecoveryCard = ({ habits, recoverableDates }) => {
  const { recoverStreak, user } = useApp();
  const [adBusy, setAdBusy] = useState<string | null>(null);

  const recoverableHabits = habits.filter(h => recoverableDates[h.id]);
  if (recoverableHabits.length === 0) return null; // don't render at all

  // ...
};
```

Each recoverable habit offers two buttons:

- **Ad button:** simulates a 3-second ad, then calls `recoverStreak(habitId, date, "ad")`
- **Coins button:** immediately calls `recoverStreak(habitId, date, "coins")` — costs 100 coins

The `recoverStreak` function in `useAppState` adds the missed date to the habit's completions array (as if it were completed that day) and either charges coins or gives a small coin bonus.

28. Component: Heatmap

File: `src/components/Heatmap.tsx`

A GitHub-style activity grid showing the past year of habit completions. Each cell represents one day, colored based on how many habits were completed.

The intensity is calculated per-day:

```
0 habits = secondary (gray)
1-25%    = primary/30 (faint green)
```

```
26-75%   = primary/60 (medium green)
76-100%  = primary    (full green)
```

The grid renders all 365 days (or days since the first habit was created) as small squares in a 7-row grid (one row per weekday).

29. Component: OnboardingFlow

File: `src/components/OnboardingFlow.tsx`

A multi-step wizard shown to first-time users:

1. **Welcome** — "Let's set up your Sprout"
2. **Name** — display name + character name
3. **Goal** — "What do you want to achieve?" (free text)
4. **Starter habits** — pick from a list of common habits with pre-filled emojis/colors
5. **Reminders** — toggle daily reminder + choose time

On completion:

```
const { completeOnboarding } = useApp();
completeOnboarding({
  displayName,
  characterName,
  goal,
  starterHabits,
  reminderEnabled,
  reminderTime,
});
```

`completeOnboarding` in `useAppState`:

1. Sets `onboardingDone: true`
 2. Sets the display name, character name, and goal
 3. Generates a unique 6-character friend code
 4. Creates all the starter habits
 5. Sets up the reminder if enabled
 6. Pushes everything to Supabase
-

30. Component: SplashScreen

File: `src/components/SplashScreen.tsx`

A branded loading screen shown on first visit (once per browser session). Displays the app logo/mascot with an animation, then calls `onDone()` after 2–3 seconds or when the animation completes.

31. Engagement Systems

The app has multiple systems designed to keep users coming back daily.

31.1 Daily Login Bonus (7-day streak)

Day	Bonus
1	5 coins
2	10 coins
3	15 coins
4	20 coins
5	25 coins
6	30 coins
7	50 coins

The streak resets if the user misses a day. This is designed to build a daily habit of *opening the app*.

31.2 Coins Economy

Coins are earned by:

- Daily login (5–50 coins)
- Completing habits (5 coins per completion)
- Watching ads (5–50 coins, random)
- Recovering a streak via ad (5–15 coins bonus)
- Completing 100% of today's habits (bonus coins)

Coins are spent on:

- Streak freeze: 50 coins
- Streak recovery: 100 coins
- Shop items: 30–250 coins

31.3 Weekly Recap (Saturday)

Every Saturday, a modal shows:

- This week's completion rate
- Up/down comparison vs last week
- Current streak
- Total habits done
- A motivational message based on performance

The message varies by rate:

- $\geq 90\%$: "You absolutely crushed it this week! 🏆"
- $\geq 70\%$: "Solid week — you showed up and delivered! 💪"

- $\geq 50\%$: "Good effort. Push for 70% next week! 🌱"
- $\geq 30\%$: "Rough weeks happen. Tomorrow is a fresh start 🌅"
- $< 30\%$: "Every champion has a comeback week. Next week is yours 🥊"

31.4 Streak Warning at 8 PM

A browser notification sent around 8 PM if any scheduled habits are incomplete. The message is randomly selected from 10 funny/dramatic options:

- "Sprout is stress-eating and it's YOUR fault 🌱😬"
- "Your streak has entered its villain arc. Redemption arc still available."
- "Your habit is doing the puppy eyes 🥺 Don't be the person who says no to puppy eyes."
- "This notification is your villain origin story... unless you go do the thing right now."

31.5 Streak Recovery System

If a user missed a habit yesterday but had recent activity in days 2–7, a recovery card appears on the home screen. This prevents the "I missed one day, why bother continuing" drop-off.

31.6 Streak Freezes

Users can spend 50 coins to "freeze" today's streak. The frozen day is added to `user.streakFreezes` and treated as "completed" in all streak calculations. Pro users would get auto-freeze (not yet implemented — this is the main Pro upsell).

31.7 Badges

Ten achievement badges unlock automatically as milestones are hit. They appear in a scrollable row on the home screen as permanent trophies.

32. Monetization Logic

32.1 Free vs. Pro Features

Feature	Free	Pro
Habit limit	5 permanent	Unlimited
AI Chat	X	✓
AI Coach	X	✓
Streak auto-freeze	X (manual with coins)	✓ (planned)
Advanced analytics	Basic	Full
Exclusive shop items	X	✓ (planned)

32.2 Free Habit Limit Enforcement

When a free user hits 5 habits and tries to create another:


```
const atLimit = !user.isPro && permanentCount >= FREE_HABIT_LIMIT && !isEdit;
```

The form is replaced with an upsell screen. Importantly, users can still create **temporary habits** (valid today only) by watching an ad. This:

1. Lets them keep using the app even at the limit
2. Demonstrates the value of unlimited habits
3. Generates ad revenue

32.3 The "Lost Streak" Loss Aversion Card

On the Upgrade page, the app shows:

```
Your streak now: 3d | With Pro: 14d
```

```
"You missed 11 days that broke your streak. Pro's auto-freeze  
would have kept it alive automatically – no coins needed."
```

Loss aversion is psychologically more powerful than gain framing. Showing users what they *already lost* is more persuasive than showing what they could gain.

32.4 Coin Scarcity

The coin system creates natural pressure to upgrade. Streak freezes (50 coins) and streak recoveries (100 coins) drain the balance, while earnings are capped. Pro removes this friction.

33. Theming System

File: `src/hooks/useTheme.tsx`

The app supports three themes: light, dark, and system (follows OS preference).

```
const ThemeProvider = ({ children }) => {  
  const { user } = useApp();  
  
  useEffect(() => {  
    const root = document.documentElement; // the <html> element  
    const theme = user.theme ?? "system";  
  
    if (theme === "system") {  
      const isDark = window.matchMedia("(prefers-color-scheme: dark)").matches;  
      root.classList.toggle("dark", isDark);  
    } else {  
      root.classList.toggle("dark", theme === "dark");  
    }  
  })  
}
```

```
    }, [user.theme]));
  };

```

Adding/removing the `dark` class on `<html>` is how Tailwind's dark mode works. All color utilities have dark variants:

```
/* Defined in CSS variables */
:root {
  --background: 0 0% 100%; /* white in light mode */
  --foreground: 240 10% 3.9%; /* near-black */
}
.dark {
  --background: 240 10% 3.9%; /* near-black in dark mode */
  --foreground: 0 0% 98%; /* near-white */
}

```

And in Tailwind: `bg-background` → `hsl(var(--background))`. This means every component automatically adapts to the theme without any conditional logic in the component itself.

34. How Data Flows End-to-End

Here is the complete flow from user action to database write:

Example: User taps the check on a habit

1. User taps HabitCard's check circle
 - ↓
2. HabitCard calls `toggleCompletion(habit.id, today)` from `useApp()`
 - ↓
3. `useAppState.toggleCompletion`:
 - a. `setHabits(prev => ...)` – optimistic update (UI changes instantly)
 - b. `saveHabits(updatedHabits)` – write to `localStorage` (synchronous)
 - c. `pushHabit(updatedHabit)` – async Supabase upsert
 - d. `setUser(newUser)` – update coin balance (+5 coins)
 - e. `pushProfile(newUser)` – async Supabase update
 - ↓
4. React re-renders HabitCard with `done=true`
 - Emoji icon scales down and grays out
 - Name gets strikethrough
 - Check circle fills green
 - Flame icon updates with new streak number
 - ↓
5. Home page re-renders
 - "done today" counter increments
 - Mascot speech updates
 - Coin balance updates
 - ↓
6. In the background (async):

- Supabase upserts the habit row with new completions array
- Supabase updates the profiles row with new coin balance
- If either fails, `warnSyncFailed()` shows a toast once

Example: App loads for a returning user

1. `main.tsx` mounts `<App />`
2. App renders `<AuthProvider>`
3. `AuthProvider` subscribes to auth changes (before `getSession`)
4. `AuthProvider` calls `getSession()`
 - `loading = true` while waiting
5. Session restores from browser storage
 - `setSession(session)` fires
 - `loading = false`
6. `RequireAuth` sees session exists → renders `AppShell`
7. `AppStateProvider` initializes:
 - `habits = loadHabits()` from `localStorage` (instant)
 - `user = loadUser()` from `localStorage` (instant)
8. Cloud load `useEffect` fires:
 - `authLoading` is now false, `authUser` exists
 - Fetches habits from Supabase
 - Merges cloud + local habits (union completions)
 - Fetches profile from Supabase
 - Merges profile fields
 - `setSyncing(false)` – spinner disappears
9. Login bonus `useEffect` fires:
 - syncing is now false
 - Checks if today's login is already recorded
 - If not: calculates streak day, adds bonus coins, shows modal
10. Weekly recap `useEffect` fires (if Saturday):
 - Shows recap modal after bonus modal is dismissed

35. Glossary

API — Application Programming Interface. A way for code to talk to external services. Supabase exposes an API that the app uses to read/write the database.

Async/Await — A pattern for handling operations that take time (like network requests). `await` pauses the current function until the operation completes, without blocking the entire app.

Component — A reusable piece of UI defined as a function that returns JSX. Equivalent to a Python function that returns an HTML string, but much more powerful.

Context — A React mechanism for sharing data across the component tree without prop drilling. Used extensively in this app via `useAppState` and `useAuth`.

Hook — A function starting with `use` that lets you use React features (state, effects, context) in function components. Custom hooks package reusable logic.

JSX — JavaScript XML. HTML-like syntax used inside JavaScript/TypeScript files. `<div className="card">` compiles to `React.createElement("div", { className: "card" })`.

localStorage — Browser storage that persists indefinitely. Key-value pairs of strings. Used as the offline cache for all app data.

Optimistic update — Updating the UI immediately as if an action succeeded, then syncing to the server in the background. Makes the app feel instant even on slow connections.

Props — Short for "properties". The parameters passed to a React component, like function arguments. `<HabitCard habit={h} date={today} />` passes `habit` and `date` as props.

Provider — A component that wraps other components to share data with them via Context.

Ref (`useRef`) — A container that holds a mutable value that persists across renders but doesn't trigger re-renders when changed. Used for: accessing DOM elements, storing previous values, avoiding stale closures.

RLS — Row Level Security. PostgreSQL feature that restricts which rows each user can see/modify. All Supabase tables in this app use RLS to ensure users only access their own data.

Stale closure — A function that "captured" old variable values when it was created, and doesn't see newer values even though they changed. The `useRef` pattern solves this.

State — Data managed by React that, when changed, triggers a re-render of components that use it. Managed with `useState`.

Supabase — A hosted PostgreSQL database with Auth, REST API, real-time subscriptions, and serverless edge functions. Used as the cloud backend for all persistent data.

TypeScript — A superset of JavaScript that adds static type checking. Code is written in `.ts/.tsx` files and compiled to plain JavaScript.

Upsert — "Update or Insert". A database operation that inserts a new row if the key doesn't exist, or updates the existing row if it does.

UUID — Universally Unique Identifier. A random 36-character string used as IDs (e.g. `"a1b2c3d4-e5f6-7890-abcd-ef1234567890"`). Every habit and user gets one.

This document covers Sprout Habits v1.0.0. All code paths described are based on the actual implementation as built.